

OPERATING SYSTEMS (UE24CS242B)

Unit 1 – Case Study – Linux & Windows Scheduling Policies

Case Study: Linux / Windows Scheduling Policies

This lecture studies **real-world CPU scheduling implementations** in two widely used operating systems:

- **Linux**
- **Microsoft Windows**

The goal is to understand how theoretical scheduling concepts are applied in practical OS designs.

Linux Scheduling Through Version 2.5

Process Scheduling in Early Linux

- Linux kernel used a variation of the **traditional UNIX scheduling algorithm**
- Limitations:
 - Did **not support SMP (Symmetric Multiprocessing)**
 - Poor performance with a large number of processes
- Scheduling decisions were inefficient for modern workloads

Linux Scheduling – Version 2.5

Major improvements were introduced:

Key Features

- Scheduler achieved **constant-time scheduling: $O(1)$**
- Added support for **SMP systems**
- Implemented:
 - Processor affinity
 - Load balancing across processors

Limitation

- Poor response time for **interactive processes**
- Not suitable for desktop systems with many short interactive tasks

Linux Scheduling in Version 2.6.23 and Later

Linux introduced the **Completely Fair Scheduler (CFS)**.

Core Ideas

- CFS is the **default scheduling algorithm**
- Based on **scheduling classes**
- Each class has a specific priority
- Kernel selects:
 - Highest priority task
 - From the highest priority scheduling class

Scheduling Classes

- **CFS scheduling class**
- **Real-time scheduling class**
- Additional classes can be added

Completely Fair Scheduler (CFS) – CPU Share Model

CFS aims to **fairly distribute CPU time** among all runnable tasks.

CPU Time Allocation

- Each task receives a **proportion of CPU time**
- Proportion is based on the **nice value**
 - Range: **-20 to +19**
 - Lower nice value → higher priority → more CPU time

Target Latency

- Time interval in which every runnable task should run at least once
- Target latency **increases as number of active tasks increases**
- CPU time slices are derived from this target latency

CFS – Virtual Runtime (vruntime)

CFS does **not assign traditional priorities**.

Virtual Runtime

- Each task maintains a **virtual runtime (vruntime)**
- Represents how much processor time the task has received
- Includes a **decay factor** based on priority:
 - Lower priority → vruntime increases faster
- Default priority:
 - $vruntime \approx \text{actual runtime}$

Task Selection

- Scheduler always selects the task with the **lowest vruntime**

CFS – Data Structure Used

Balanced Binary Search Tree

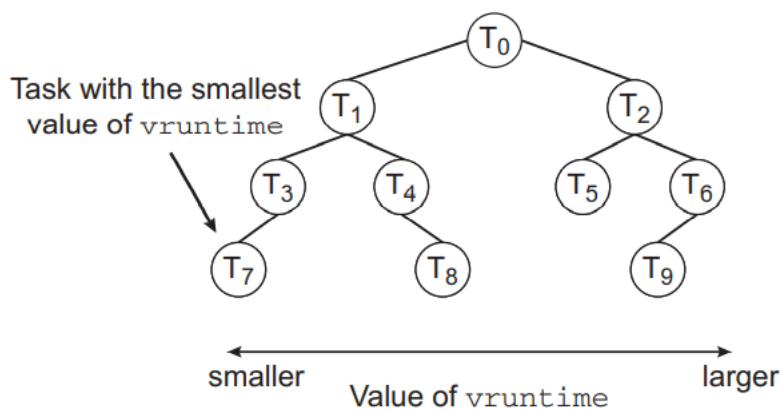
- All runnable tasks are stored in a **balanced binary search tree**
- Key = vruntime

Operations

- Task becomes runnable → inserted into tree
- Task blocks → removed from tree
- Scheduler selects **leftmost node** (smallest vruntime)

Time Complexity

- Selection requires **$O(\log N)$** time (N = number of runnable tasks)



CFS – I/O-Bound vs CPU-Bound Processes

Assume:

- Two tasks have the same nice value
- One is **I/O-bound**
- Other is **CPU-bound**

Behavior

- I/O-bound task accumulates **less vruntime**
- CPU-bound task accumulates **more vruntime**
- Result:
 - I/O-bound task gets **higher priority**
 - If I/O-bound task becomes runnable, it **preempts** CPU-bound task

This improves **interactive responsiveness**.

Linux Real-Time Scheduling

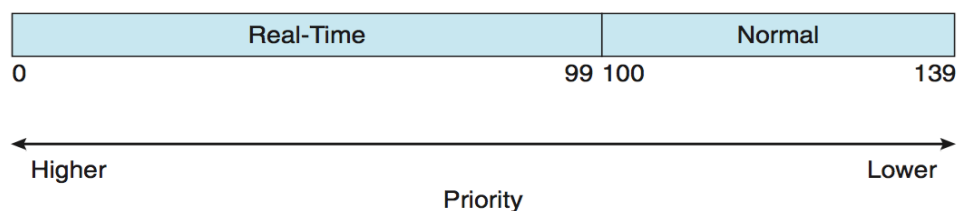
Linux supports **POSIX.1b real-time scheduling**.

Key Characteristics

- Real-time tasks have **static priorities**
- Priority range: **0 to 99**
- Real-time tasks always preempt normal tasks

Global Priority Mapping

- Nice value $-20 \rightarrow$ global priority 100
- Nice value $+19 \rightarrow$ global priority 139
- Real-time priorities exist above normal tasks



Windows Scheduling

Windows uses a **priority-based preemptive scheduling algorithm**.

Core Principles

- Highest-priority **thread** always runs
- Scheduling handled by the **dispatcher**
- Selected thread runs until:
 - Preempted by higher-priority thread
 - Terminates
 - Time quantum expires
 - Calls a blocking system call

Windows Priority Scheme

Priority Levels

- **32 priority levels**
- Priority 0 $\rightarrow$ reserved for memory management

- Priority 1–15 → Variable class
- Priority 16–31 → Real-time class

If no runnable thread exists:

- **Idle thread** executes

Each priority level has its **own ready queue**.

Windows Priority Classes

Windows API defines the following priority classes:

- REALTIME_PRIORITY_CLASS
- HIGH_PRIORITY_CLASS
- ABOVE_NORMAL_PRIORITY_CLASS
- NORMAL_PRIORITY_CLASS
- BELOW_NORMAL_PRIORITY_CLASS
- IDLE_PRIORITY_CLASS

Notes

- All are **variable priority classes**, except REALTIME
- Each thread also has a **relative priority**:
 - TIME_CRITICAL
 - HIGHEST
 - ABOVE_NORMAL
 - NORMAL
 - BELOW_NORMAL
 - LOWEST
 - IDLE

Windows Thread Priorities

A thread's **numeric priority** is determined by:

- Process priority class
- Thread's relative priority within the class

Base Priority

- Base priority is **NORMAL** within the class
- If time quantum expires:

- Priority may be lowered
- Never reduced below base priority

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Windows Priority Boosting

I/O Completion Boost

- If a thread wakes up after waiting:
 - Priority may be temporarily boosted
- Boost depends on:
 - What the thread was waiting for

Foreground vs Background

- Windows distinguishes foreground processes
- Foreground process gets **3× priority boost**
- This allows:
 - Foreground application to run longer
 - Better user responsiveness